

PyTorch compile in the wild

By: **Ayush Satyam**
Associate MLE at  **Red Hat**

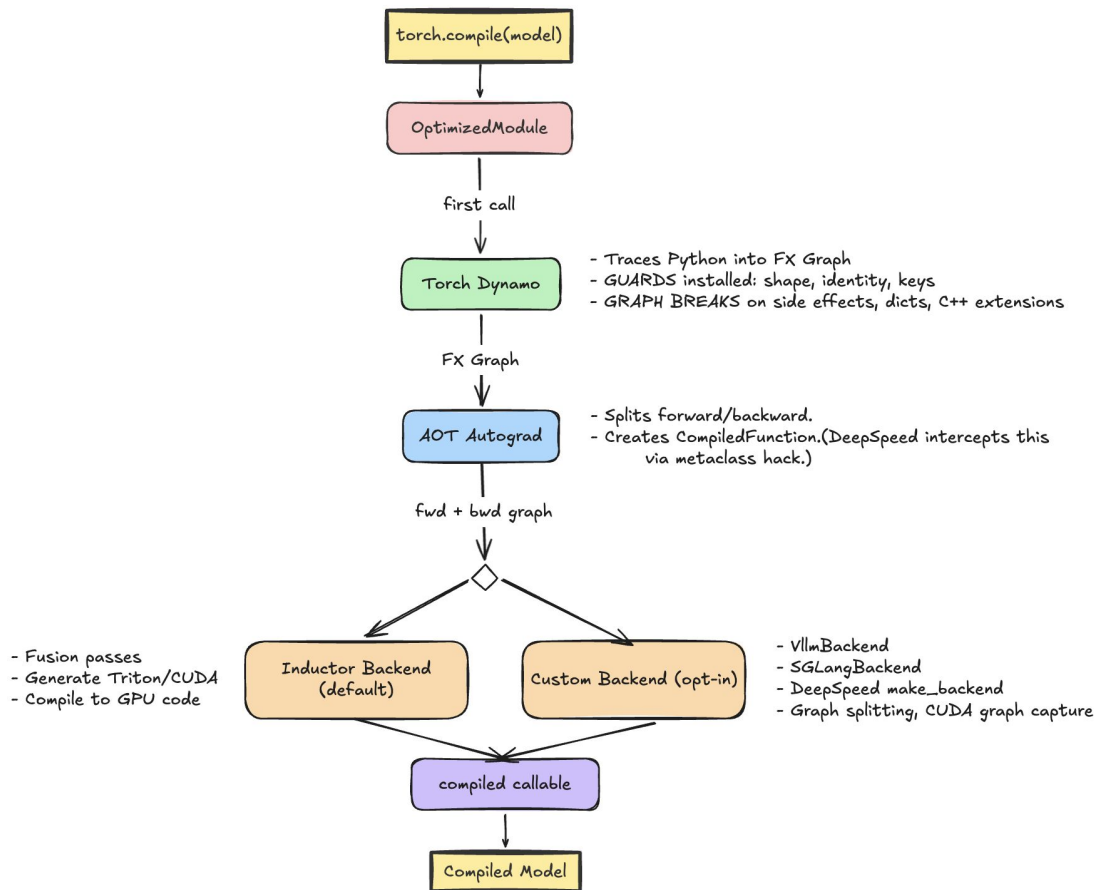
12 projects, **5 domains** all around torch.compile



Pytorch **ecosystem** and torch.compile

- **What happens on `model = torch.compile(model)`**
- **Every domain has different challenges to overcome**
- **Compilation needs and boundaries are different**
 - **Inference cares about compile time**
 - **Training cares about distributed wrappers**
 - **RL can't even compile training loop**
 - **GNN's have to generate code at import time just to avoid graph breaks**

A refresher of **torch.compile**



Projects categorised by **use-cases**

Inference

 vLLM

 Transformers

 SGL

 Diffusers

Domain-specific



MONAI⁺

Distributed

 DeepSpeed

 PyTorch
Lightning

Training

 Accelerate

 PyTorch
Lightning

Reinforcement
learning

 TorchRL

ACT 1 - INFERENCE

The **Inference** Constraint?

“Compile time matters as much as runtime”

- Compilation time is mainly a cold-start cost
- Inference workloads usually have mostly static shapes, making them ideal for torch.compile.
- Two key challenges in production inference:
 - deciding what parts to compile
 - handling adapter swapping efficiently.
- Two levels of inference
 - Library level (Transformers and diffusers)
 - Framework level (vLLM and SGL)



Transformers **auto compile** during generate(...)

When you call generate(), it automatically decides whether to compile or not based on:

- Device = CPU | XPU??
- Is cache compilable? Disk offloading enabled?
- Using flash attention or not (it forces fullgraph=false)

So what's the **catch**??

They only compile the DECODE step: Prefill is variable length and runs once. Decode is fixed shape and runs N times. Compiling prefill is wasted work.

PREFILL: variable length, runs once	NOT compiled
DECODE LOOP: fixed shape, runs N times	auto-Compiled



Diffusers: **fragmented, regional** compilation

Diffusers go a little further. They never compile the full pipeline.

They don't even compile the full model. They compile only the repeated blocks:

```
# transformer_flux.py
_repeated_blocks = ["FluxTransformerBlock", "FluxSingleTransformerBlock"]

# modeling_utils.py
def compile_repeated_blocks(self, *args, **kwargs):
    for submod in self.modules():
        if submod.__class__.__name__ in repeated_blocks:
            submod.compile(*args, **kwargs) # compile each block type ONCE
```

25 models in Diffusers define `_repeated_blocks`. The compiled artifact is reused for every occurrence of the same block class.

If your model **has N identical layers**, this is **free performance**.

Embedding layer	NOT compiled non-repititive layer
FluxTransformerBlock [0] FluxTransformerBlock [1] FluxTransformerBlock [2] FluxTransformerBlock [3] ... FluxTransformerBlock [23] reused 24x	Compiled compile(...) called once resued 24x
FluxSingleTransformerBlock [0] FluxSingleTransformerBlock [1] ... FluxSingleTransformerBlock [11]	Compiled same idea different block type
Output projection	NOT compiled

Diffusers: LoRA hotswap without re-compilation!!

Phase A: PREPARATION

```
enable_lora_hotswap(target_rank=128)
```

Pre-allocate LoRA matrices at MAX rank:

LoRA_A: [in_features x 128] <-- padded to 128

LoRA_B: [128 x out_features] <-- even if rank < 128

Phase B: FIRST ADAPTER + COMPILE

```
load_lora_weights("adapter_1.safetensors")  
pipe.transformer = torch.compile(pipe.transformer)
```

Shapes locked: LoRA_A = [in x 128]

LoRA_B = [128 x out]

Compiled graph guards on these shapes.

Phase C: HOTSWAP -- NO RECOMPILATION

```
load_lora_weights("adapter_2.safetensors", hotswap=True)
```

adapter_2 has rank 32?

-> pad to 128, in-place param.data.copy_()

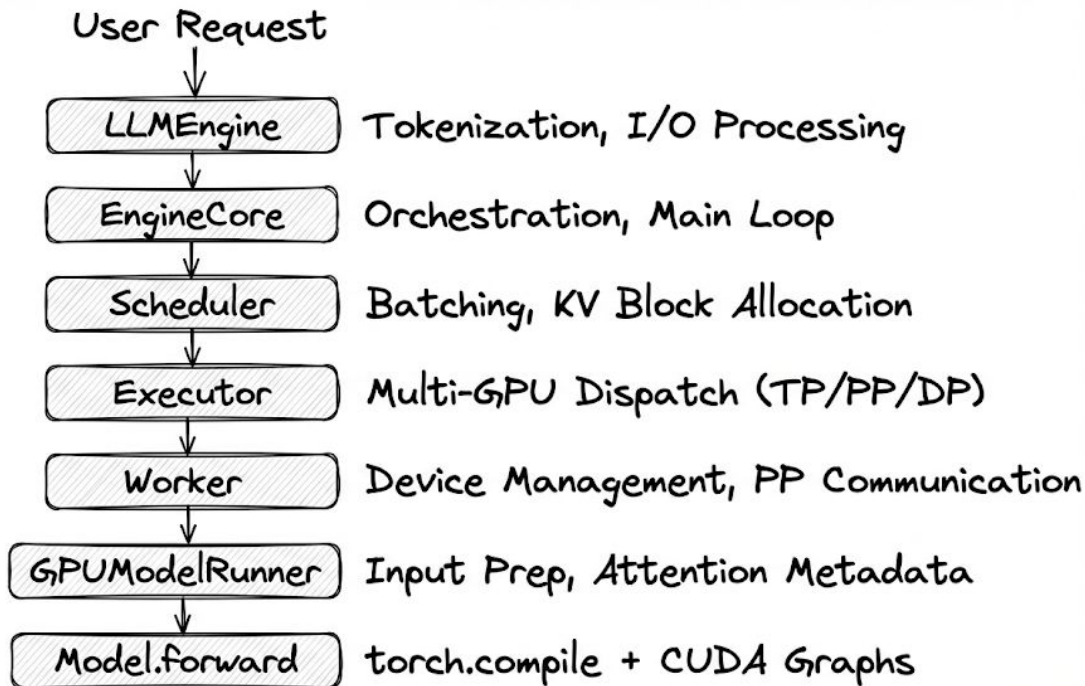
-> same tensor identity, same shape, same dtype

-> torch.compile guards pass

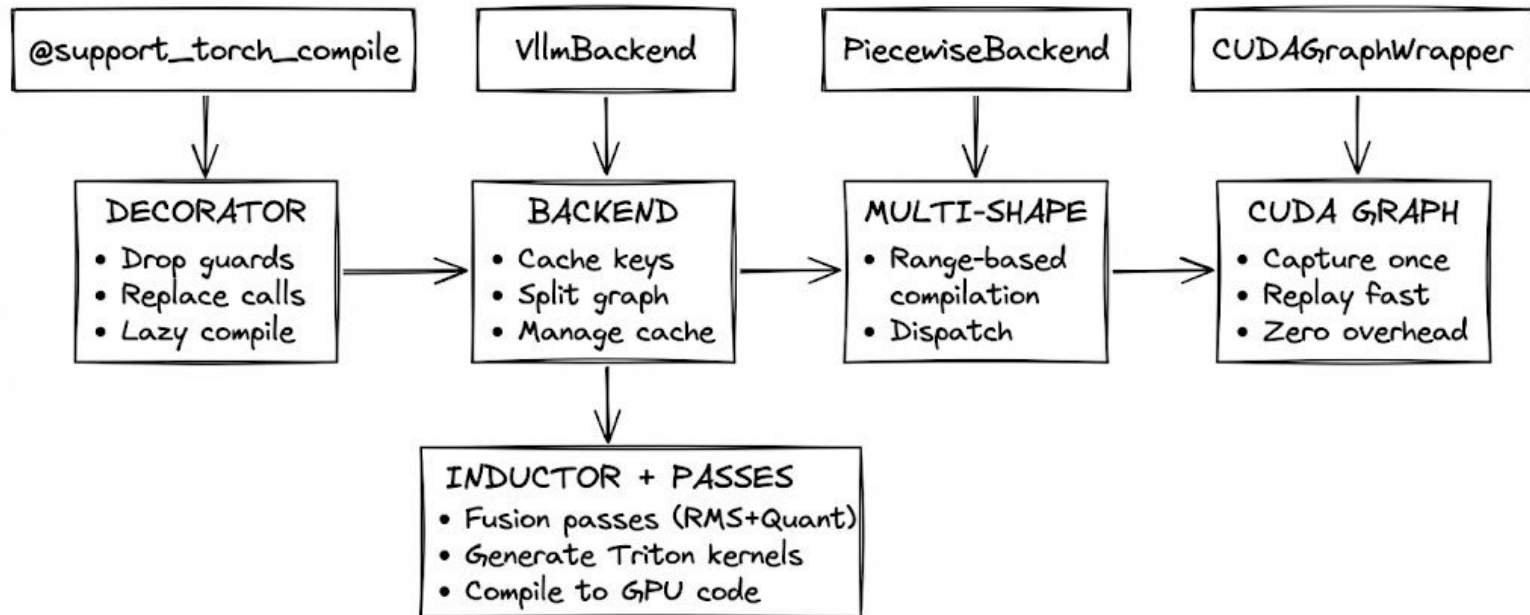
-> ZERO recompilation

vLLM : handling compile via **custom backend**

where torch.compile meets vLLM



VLLM COMPILATION PIPELINE

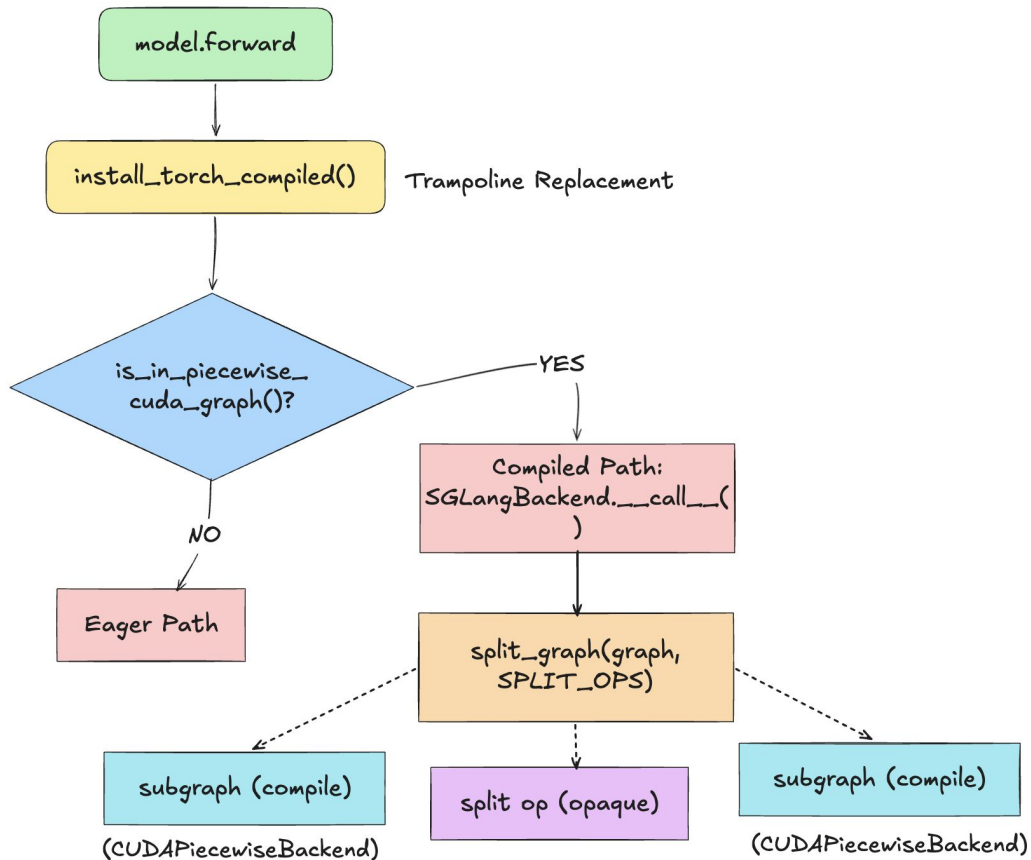


SGL : improvising on vLLM's approach

SGLang took vLLM's approach and evolved it. Their **SGLangBackend** is adapted from vLLM's codebase, So what's different?

- It splits graphs at **attention + all-reduce ops**, also any model can register its **own split points**.
- Compile dimension. vLLM **captures CUDA graphs by batch size**. SGLang captures by token count. PD operations both supported
- vLLM always uses Inductor for the compilable subgraphs. SGLang **defaults to eager + CUDA graph replay**; Inductor is opt-in.

SGL compilation architecture



ACT 2 - DISTRIBUTED TRAINING

The **Distributed Training** Constraint?

“Distributed training works by side effects. Compile works by eliminating them.”

- Your model gets wrapped - **DDP**, **FSDP**, **mixed precision**. And the wrapping **ORDER** matters.
- These wrappers use **hooks** which are callbacks that fire during forward/backward to **sync gradients** and **gather sharded parameters**.
- `torch.compile` **can't trace** hooks
- Two key challenges in production training:
 - Wrapper ordering
 - hooks: biggest reason for graph breaks
- Two approaches
 - Reorder the wrappers
 - Strip hooks, inject custom ops in graph



Accelerate: solved the `_call_impl` problem

HF Accelerate fixes a subtle issue: `torch.compile(model)` wraps models in `OptimizedModule`, replacing `__call__` and bypassing `nn.Module._call_impl` (the hook point used by FSDP and DeepSpeed)

`torch.compile(model)` returns `OptimizedModule`:

`OptimizedModule.__call__()`

- > dynamo tracing
- > calls `_orig_mod.forward()` directly

- Bypasses `_call_impl`
- FSDP hooks never fire

`module.compile()` modifies in-place:

`model.__call__()`

- > `nn.Module._call_impl()`
 - > FSDP `pre_forward` hook (all-gather)
 - > compiled `forward()`
 - > FSDP `post_forward` hook (reshard)

Hooks still fire!

Accelerate maintains THREE compilation strategies - one per distributed backend

```
# Standard DDP: torch.compile() wraps AFTER DDP (compile sees all-reduce)
def compile_regions(module, **compile_kwargs):
    # uses torch.compile(submodule) -- returns OptimizedModule wrapper
    new_module.append(torch.compile(submodule, **compile_kwargs))

# FSDP2: module.compile() BEFORE fully_shard (hooks register after)
def compile_regions_fsdp2(module, **compile_kwargs):
    # uses module.compile() -- in-place, preserves _call_impl
    submodule.compile(**compile_kwargs)

# DeepSpeed: module.compile() AFTER engine creation (hooks already exist)
def compile_regions_deepspeed(module, **compile_kwargs):
    # uses module.compile() -- avoids interfering with offloaded params
    submodule.compile(**compile_kwargs)
```

The Distributed Dilemma?

"The order you wrap is the order you make or break"

The fundamental dilemma: **Distributed** training uses hooks, and **torch.compile** traces **Python into a graph**. Hooks are **invisible** to the **graph**

Distributed hooks:

```
pre_forward_hook:
  gather_params
post_backward_hook:
  reduce_grads
post_forward_hook:
  release_params
```

vs.

torch.compile

Traces Python into a static
FX graph

Hooks = opaque side effects
= GRAPH BREAKS

Two choices: **reorder the wrappers** so compile can see through the distributed layer, or **strip the hooks entirely** and inject ops into the graph. **Lightning** chose the first. **DeepSpeed** chose the second.

PyTorch Lightning: just replaces **everything**

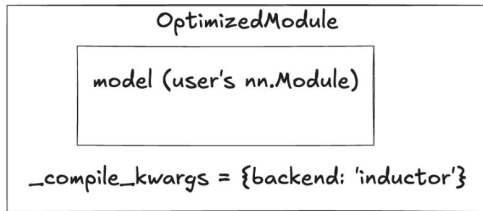
It **replaces torch.compile** globally. The Fabric wrapper intercepts every call and stashes the compile kwargs on the model as `model._compile_kwargs`.

the USER writes:

```
model = torch.compile(model, backend="inductor")
```

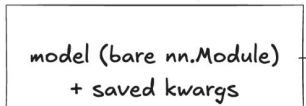
Result: compiler sees and optimizes
all-reduce ops

Step 1: User's `torch.compile` call (monkeypatched)

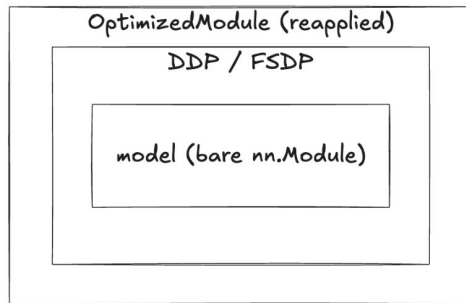


Fabric.setup()
intercepts

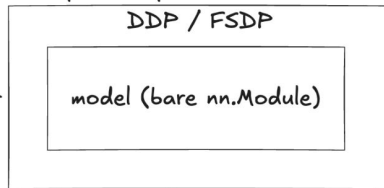
Step 2: STRIP the OptimizedModule



Step 4: REAPPLY `torch.compile` OVER DDP/FSDP



Step 3: Wrap with DDP/FSDP





DeepSpeed: made its own compile framework

DeepSpeed didn't just use `torch.compile`. They built a **30-file sub-framework** ON TOP of it called **DeepCompile**.

The most audacious tweaks of DeepCompile:

- They strip ALL standard ZeRO-3 hooks and inject custom ops into the compiled graph instead:
- Schedule-based recompilation. At step 0, compile with basic ops. Run 5 warmup steps to profile.

At step 5, call `torch._dynamo.reset()` to force recompilation with profiling-guided optimizations. Recompilation is cheap relative to training.

- They need to intercept the backward pass inputs from AOT Autograd's internal Compiled function. These **aren't exposed**. So they **replace** `torch.autograd.Function`'s **METAClass**:

```

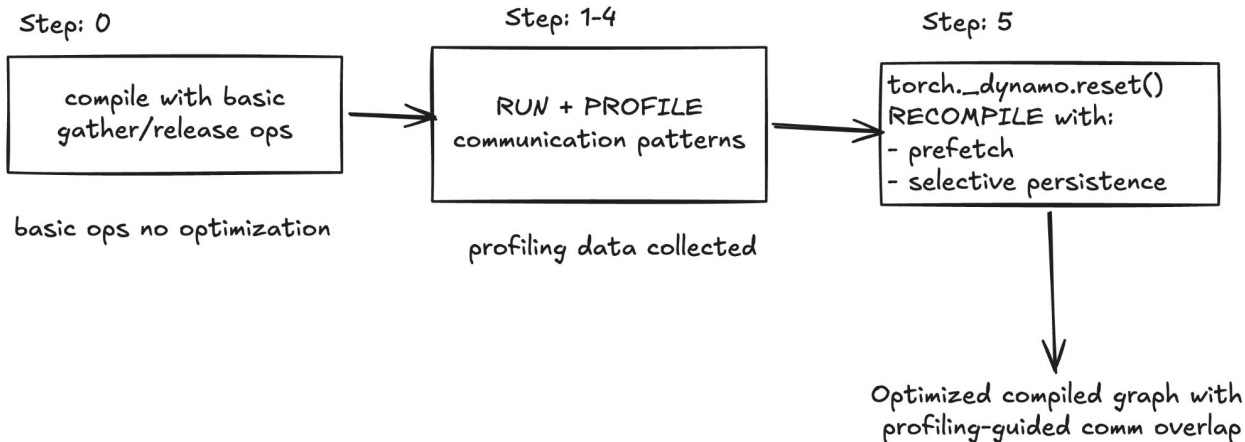
# The MetaClass hack
# compile/patch_compiled_func.py
class FunctionMeta(base_meta):
    def __new__(cls, name, bases, dct):
        if name == "CompiledFunction":          # intercept BEFORE the class exists
            original_backward = dct["_backward_impl"]
            def wrapped(ctx, all_args):
                backward_inputs.append(all_args)    # capture real backward inputs
                return original_backward(ctx, all_args)
            dct["_backward_impl"] = staticmethod(wrapped)

torch.autograd.Function = PatchedFunction          # global replacement

```

**The Metaclass hack
for AOT Autograd's
internal Compiled
function**

Schedule-based recompilation.



ACT 3 - REINFORCEMENT LEARNING

The **RL** Constraint?

“You can’t compile the training loop. So what do you even compile?”

- RL training loops include **env.step()**, arbitrary Python with side effects, variable length episodes, random resets.
- The **environment interaction** is completely **untraceable** by Dynamo
- **You can’t compile the loop** So what DO you compile?
- Two key challenges in production training:
 - **env.step() is untraceable**, you can’t compile the training loop
 - data-dependent shapes: trajectory **lengths vary**, torch.where() returns variable-count indices
- Two approaches
 - **Compile only the update** function (TorchRL)
 - Make the **container type compile friendly** (TensorDict)

TorchRL : Compile the math, not the loop. They wrap loss + backward + optimizer step as one compiled callable:

RL Training loop

```
for each iteration:  
    env.step(action)  
    env.reset()  
    replay_buffer.sample()  
    env.render()  
    episode bookkeeping
```

NOT compiled

```
update(sampled_tensordict):  
    loss_td = loss_module(td)  
    (actor + q + alpha).sum().backward()  
    optimizer.step()  
    optimizer.zero_grad(set_to_none=True)  
    target_net_updater.step()  
    return loss_td.detach()
```

Compiled
(with warmup)

The environment interaction stays in eager Python. Only the math gets compiled. In **compile_with_warmup**, the first few iterations often have non-representative shapes. Run N iterations eagerly first, THEN compile on representative data:

```
def update(sampled_tensordict):
    loss_td = loss_module(sampled_tensordict)
    (actor_loss + q_loss + alpha_loss).sum().backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
    target_net_updater.step()
    return loss_td.detach()

update = compile_with_warmup(update, mode=compile_mode, warmup=1)
```

```
count = -1; compiled_model = model
def count_and_compile(*args, **kwargs):
    nonlocal count, compiled_model
    count += 1
    if count == warmup:
        compiled_model = torch.compile(model, *args, **kwargs)
    return compiled_model(*args, **kwargs)
```

THE DATA-DEPENDENT SHAPE PROBLEM

The Problem:

Trajectory data: [...
 [^] [^] [^]
 variable variable variable
 length length length

`torch.where(done)[0] --> indices: [3, 8, 11]` <-- DATA-DEPENDENT count!
Dynamo can't handle this.

TensorDict: “Making a container **compiler-friendly**”

TensorDict is a very useful data container in TorchRL.

Making a **dict-like container work with compile** required pytree registration, bulk dict.update() to avoid DICT_KEYS_MATCH guard explosions, allow_in_graph on C++ ops, and is_compiling() dual-path guards throughout the codebase.

If you’re building a **container type** that needs to **work with compile**, **TensorDict is your reference implementation.**

ACT 4 - DOMAIN-SPECIFIC

The **Domain-specific** Constraint?

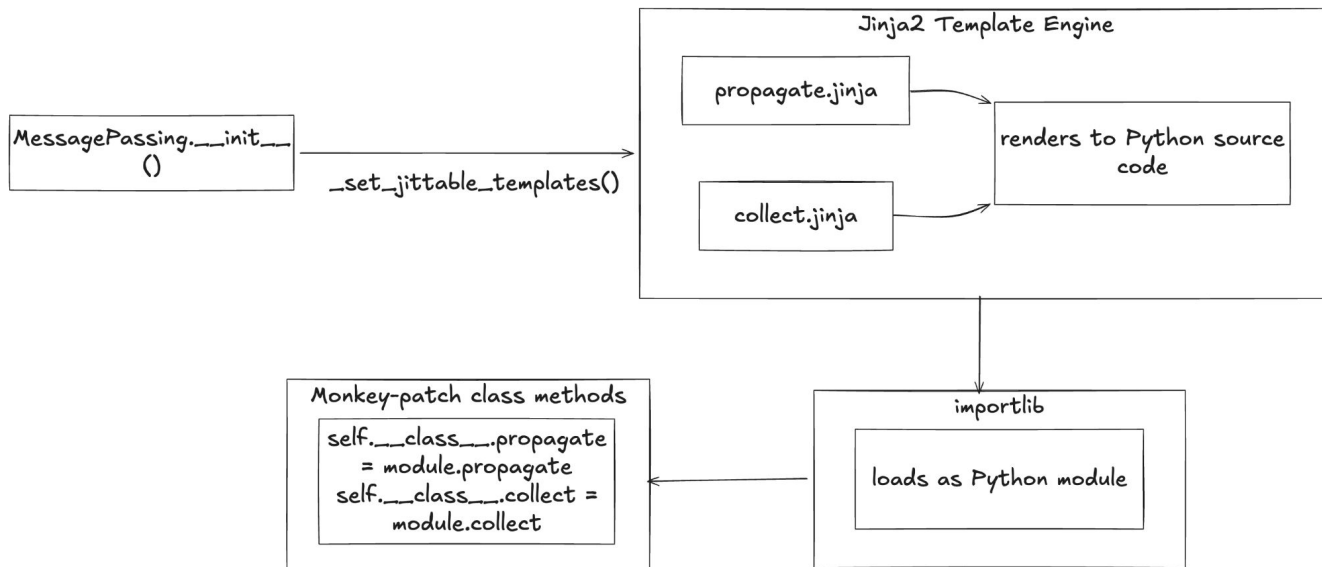
"Custom ops and tensor subclasses are where compile support drops off"

- Every domain has **ops that compiler can't see** through: C++ scatter extensions, custom CUDA kernels, ROI Pooling, NMS
- **Tensor subclasses** carry domain **metadata** that Dynamo can't handle
- All compiler needs to know is **shape contract**, not the **implementation**
- Three key challenges in production training:
 - C++ extensions can't be traced, need to fallback
 - Tensor subclasses with `__torch_function__` overrides break compile
 - Data dependent output shapes
- Three approaches
 - **Strip metadata** compile plain tensors
 - Generate compiler friendly code at import time
 - Unbacked SymInts for data-dependent shapes



PyG: The code generator

They use a very creative approach. The message-passing base class uses **Jinja2 templates** to generate compile-friendly code **AT IMPORT TIME**:



NamedTuples have **fixed fields**. **Dynamo** loves them. **Dicts** with **dynamic keys** cause **graph breaks**.

```
# BEFORE
kwargs = {}
kwargs["edge_index_i"] = edge_index[0]
kwargs["edge_index_j"] = edge_index[1]

# dict = graph break!

# AFTER
CollectArgs = NamedTuple("CollectArgs", [("edge_index_i", Tensor),
                                           ("edge_index_j", Tensor), ...])
# NamedTuple = Dynamo loves this!
```



and



Two projects, two very different approaches: making domain-specific ops and data types work with compile.

Torch-vision: make the op compile-compatible:

- `create_unbacked_symint()`: a symbolic integer the compiler can reason about without knowing the concrete value
- Pure Python ROI Align: too slow in eager, but compiled it matches the CUDA kernel
- Bonus: its backward is deterministic (the CUDA kernel's isn't)
- `lazy_compile` decorator: compiles on first call, replaces itself in `globals()`, zero overhead after

MONAI - strip what compile can't handle, reattach after:

- `MetaTensor`: tensor subclass carrying medical imaging metadata (affine transforms, voxel spacing, spatial orientation) `torch.compile` can't handle `__torch_function__` overrides
- Solution: strip before, reattach after

The **tensor subclass** spectrum

WORKS (with effort)

TensorDict

- pytree register
- 60+ dual-path guards
 - bulk dict ops
 - allow_in_graph

WORKAROUND (loses info)

MONAI MetaTensor

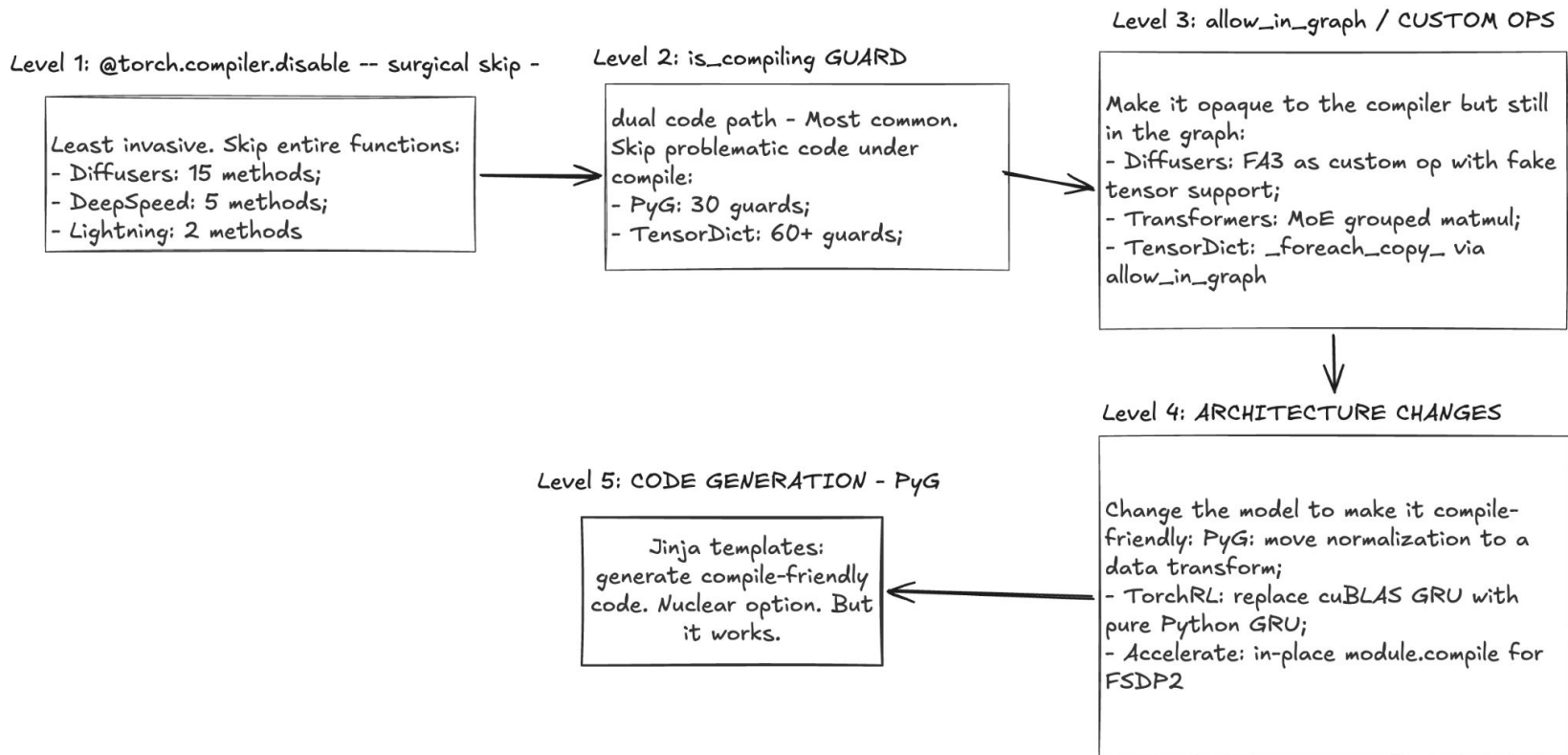
- strip metadata before compile
- run as plain tensors
 - re-attach after

BROKEN (skipped)

PyG EdgeIndex

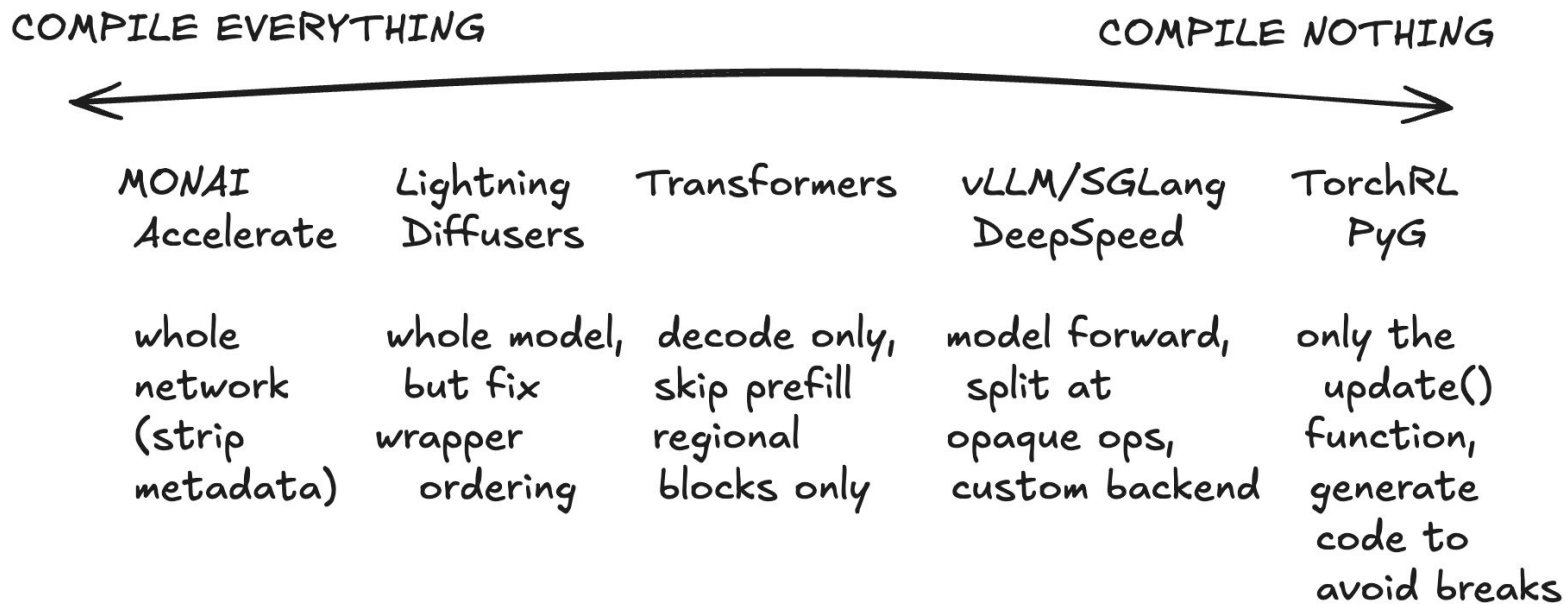
Both compile tests are
@pytest.mark.skip
reason='Does not work currently'

The **Survival** taxonomy



The **Compilation boundary** spectrum

One thing jumped out across all 12 projects. Every project draws the compilation boundary at a different point. Here's the full spectrum:



Lessons to learn

How to structure your code

- **Compilation Boundary** - Compile tensor math, not orchestration. Split at the boundary between pure ops and Python side effects.
- **Deferred Compilation** - Compile only after shapes stabilize, caches warm up, and lazy state initializes.
- **Shape Stabilization** - Keep shapes static where possible: lock shapes (LoRA hotswap), isolate dynamic paths, or dispatch by shape (vLLM/SGLang).

How to handle the untracable

- **Opaque Wrapping** - Use `custom_op`, `allow_in_graph`, or `register_fake` to expose shape contracts without exposing implementation details.
- **Dual Execution Mode** - Treat compile as a second execution path. Guard with `is_compiling()` and expect many compile-specific branches.
- **Strip-Transform-Reapply** - Remove hooks, wrappers, or metadata before transformation; restore them afterward.

How to go deeper

- **Graph Splitting** - Split around opaque ops, compile independent regions, then CUDA-graph each piece (vLLM/SGLang).
- **Compiler Monkeypatching** - Patch `torch.compile`, metaclasses, or Inductor caches when framework internals need custom behavior.
- **Progressive Compilation** - Compile incrementally: start simple, profile, reset, then recompile with better runtime information.

The Conclusion

torch.compile is not a flag you just turn on. It's an **architectural decision** that reshapes your codebase. And the projects that get the most from it are the ones **willing to meet it halfway**.

THANK YOU

Reach out to me on:



@ayushsatyam146



@ayushsatyam146

